

Kubernetes and Cloud Native Associate - KCNA

Kubernetes Resources

Pods with YAML

Hello, and welcome to this tutorial on creating a Pod using a YAML-based configuration file. In this guide, you will learn how to write and apply YAML files specifically tailored for Kubernetes. Kubernetes leverages YAML files to create various objects like Pods, ReplicaSets, Deployments, and Services. Each definition file adheres to a common structure containing four top-level fields: `apiVersion`, `kind`, `metadata`, and `spec`.

Understanding the Key Fields

Every Kubernetes YAML definition file must include the following properties:

1. Note

The four main properties in every file are: **`apiVersion`**, **`kind`**, **`metadata`**, and **`spec`**. These are essential for Kubernetes to correctly parse and create objects.

1. `apiVersion`

This property specifies the version of the Kubernetes API you are using to create your object. For Pods, you typically specify `"v1"`. For other objects like Deployments, you might use `"apps/v1"` or `"extensions/v1beta1"`.

2. kind

The `kind` field defines the type of object being created. In this lesson, we will focus on creating a Pod, so you would set `kind` as `"Pod"`. Other common values include ReplicaSet, Deployment, or Service.

3. metadata

The `metadata` section contains identifying information about the object, such as its name and labels. It is a dictionary rather than a string. Ensure all keys (like `name` and `labels`) have proper and consistent indentation. For example:

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
```

In this snippet:

- `name` uniquely identifies the Pod.
- `labels` are key-value pairs used to categorize and manage Pods based on roles such as front-end, back-end, or database.

4. spec

The `spec` section provides detailed configuration for the object. For a Pod, this involves specifying one or more containers that will run within it. The key `containers` holds a list of container definitions; each item in the list is a dictionary containing that container's configuration.

Below is a complete example of defining a Pod using YAML:

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: front-end
spec:
  containers:
    - name: nginx-container
      image: nginx
```

In this configuration:

- The `containers` key defines a list of container specifications.
- The dash before `name` indicates the first (and only) container in the list.
- The container is named `nginx-container` and it uses the Docker image `nginx`.

Deploying Your Pod

After creating your YAML configuration file (e.g., `pod-definition.yml`), you can deploy the Pod using the following command:

```
kubectl create -f pod-definition.yml
```

Kubernetes will then create the Pod as defined in your file.

Checking Pod Status

To verify that your Pod is running, use:

```
kubectl get pods
```

The output should resemble:

NAME	READY	STATUS	RESTARTS	AGE
myapp-pod	1/1	Running	0	20s

To retrieve detailed information about the Pod, execute:

```
kubectl describe pod myapp-pod
```

This command displays comprehensive details including creation time, labels, container status, and events. An example output might look like:

```
Name:          myapp-pod
Namespace:     default
Node:          minikube/192.168.99.100
Start Time:    Sat, 03 Mar 2018 14:26:14 +0800
Labels:        app=myapp
               name=myapp-pod
Annotations:   <none>
Status:        Running
IP:            172.17.0.24
Containers:
  nginx:
    Container ID:  docker://830bb56c8c2a86b4b70e9c1488fae1bc38663e491b6c2f5a783e
    Image:         nginx
    Image ID:      docker-pullable://nginx@sha256:4771d095787c6a605299e110b3ee1e
    Port:         <none>
    State:        Running
      Started:    Sat, 03 Mar 2018 14:26:21 +0800
    Ready:        True
    Restart Count: 0
    Environment:  <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from default-token-x95w7 (ro)
Conditions:
  Type            Status
  -----
```

```

    Initialized      True
    Ready            True
    PodScheduled     True
Events:
  Type    Reason              Age           From                    Message
  ----    -
  Normal  Scheduled            34s           default-scheduler      SuccessfulMountVolume
  Normal  SuccessfulMountVolume 34s           kubelet, minikube      pulling
  Normal  Pulling              27s           kubelet, minikube      Success-
  Normal  Pulled               27s           kubelet, minikube      Created
  Normal  Created              27s           kubelet, minikube      Started
  Normal  Started              27s           kubelet, minikube

```

Summary

In summary, when creating a YAML file for Kubernetes, always include the four top-level properties:

- **apiVersion:** Specifies the API version.
- **kind:** Defines the type of Kubernetes object.
- **metadata:** Holds essential information such as name and labels.
- **spec:** Details the configuration specifics, such as container settings in a Pod.

This lesson demonstrated how to create a Pod using YAML. The same principles apply when defining other Kubernetes objects such as Deployments, Services, and ReplicaSets. For more information, explore the [Kubernetes Documentation](#).



Next Steps

Consider experimenting with more complex configurations and exploring labels and selectors for robust application management in Kubernetes.



Watch Video

Watch video content

Previous

◀ Demo Pods

Next

Demo Pods with YAML ▶



Beta



● Search docs...



Kubernetes and Cloud Native Associate - KCNA ▼